

FFT Accelerator

Group 9

18-3-2026





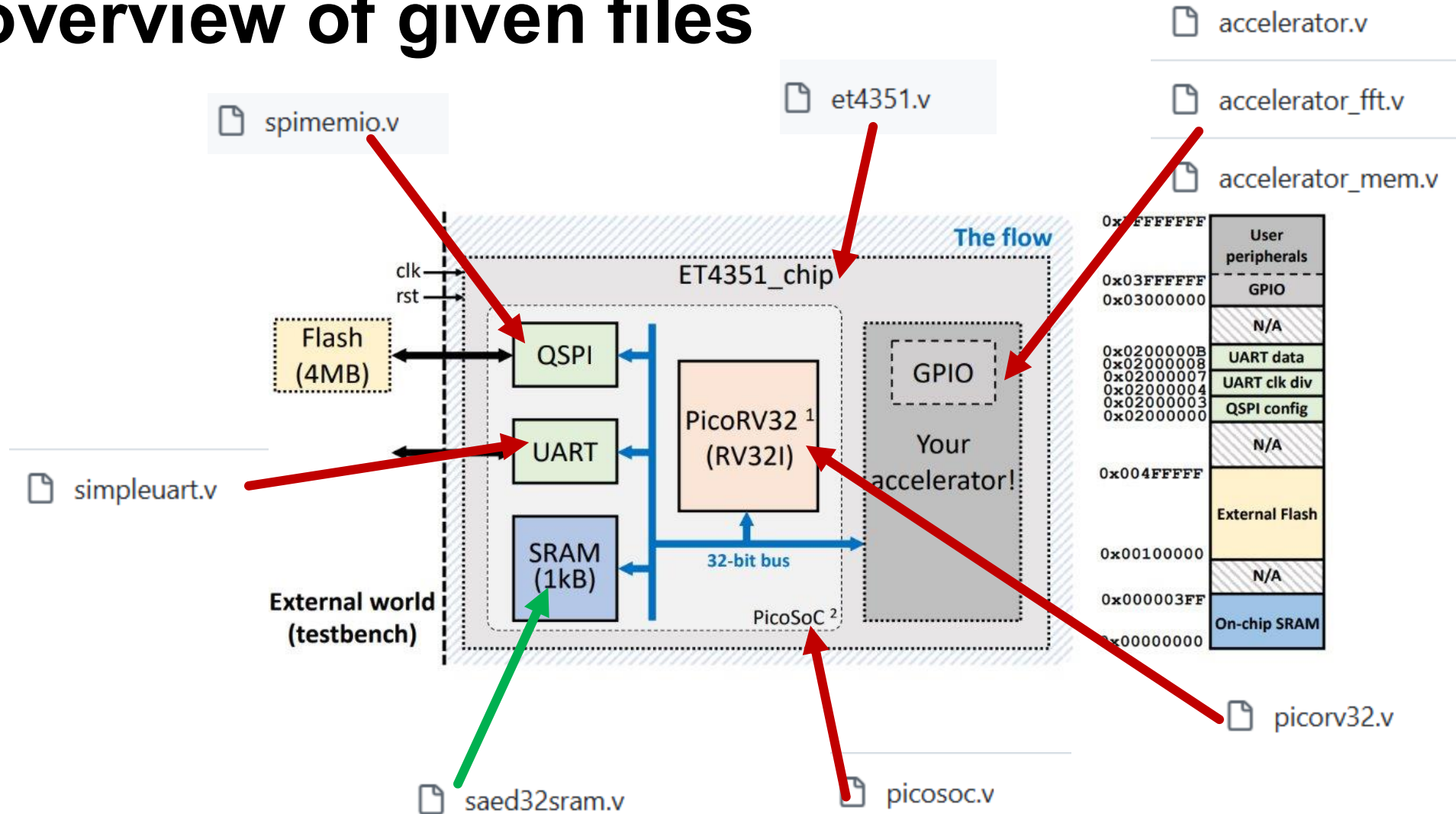
Part I – High Performance

Shanghong Lin, Alessandro Cagnacci,
Yaonan Hu, Leonardo Castello,
Anastasis Kazakos

01

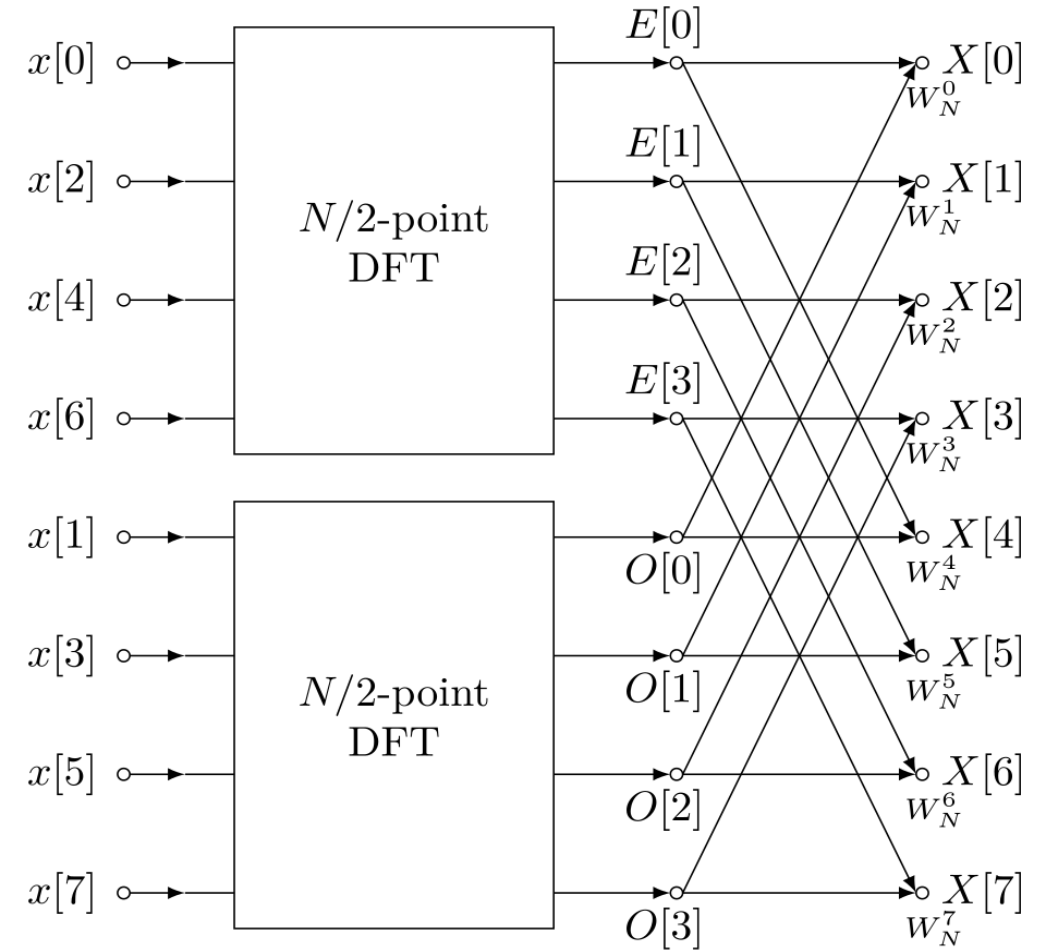
Baseline Arch & Profiling

An overview of given files



Radix-2 algorithm characteristics

- Requires power-of-2 input size
 - $N = 2^m$.
- Number of stages:
 - $\log_2(N)$.
- Number of twiddle factors:
 - $N/2$ unique twiddle factors
- Operations per stage:
 - $N/2$ in-place butterflies per stage.



Baseline overview

Single butterfly: 9 cycles

Cycle offset	State	Category
+0	BUTTERFLY_READ_1_RE	Memory read
+1	BUTTERFLY_READ_1_IM	Memory read
+2	BUTTERFLY_READ_2_RE	Memory read
+3	BUTTERFLY_READ_2_IM	Memory read
+4	BUTTERFLY_COMPUTE	Compute
+5	BUTTERFLY_WRITE_1_RE	Memory write
+6	BUTTERFLY_WRITE_1_IM	Memory write
+7	BUTTERFLY_WRITE_2_RE	Memory write
+8	BUTTERFLY_WRITE_2_IM	Memory write + loop update

6.1 Per-stage accounting

Stage	Twiddle load	Butterfly cycles (16 × 9)	Stage total
1	2	144	146
2	2	144	146
3	2	144	146
4	2	144	146
5	2	144	146
Subtotal	10	720	730
INIT	—	—	1
FINISH	—	—	1
Grand total			732

Bottleneck of the baseline

- Severely memory-bound design
 - 88.8% of all cycles are spent on SRAM reads/writes
- Root causes:
 - Scalar memory port bottleneck
 - Butterfly touches **8 words but computes in 1 cycle**
 - No pipelining
 - No data reuse (u & v from SRAM)

Category	Cycles	%
Memory access (reads + writes)	650	88.8%
Compute	80	10.9%
Control overhead	2	0.3%

02

Exploration Journey

Exploration 1: Register File Architecture

3.33× Faster

732 → 220 cycles

18.3 μ s

Latency @ 12 MHz

Drop-in

Zero FW changes

Key Idea

Replace per-butterfly SRAM access with a register file (64 data + 10 twiddle regs). Front-load all reads, then compute the full 5-stage butterfly without external memory access.

Architecture Changes

- 6-state FSM: INIT → LOAD_TW → LOAD_DATA → COMPUTE → STORE → FINISH
- Register files holds all 32 complex samples + 5-stage twiddles
- Single-cycle combinational butterfly: reg read → 32×32 multiply → shift → add/sub → write-back
- Eliminates 9-state interleaved read/compute/write from baseline design

Insight: Save cycles for intermediate data transferring, which can also be used by in-place computing of butterfly.

Exploration 2: 2× Parallel Butterflies

3.55× Faster

732 → 206 cycles

80 → 66

Compute cycles (−17.5%)

308K Area

71.3% accelerator

Key Idea

Add a second independent butterfly datapath executing simultaneously.

Architecture Changes

- COMPUTE phase split: FILL (twiddle table) + BUTTERFLY (2× parallel units)
- $N/2$ butterflies processed in pairs → $N/4$ cycles per stage (halved)
- Per-stage twiddle fill: $tw[k] = tw[k-1] \times \text{prim}[\text{stage}]$, bit-exact with reference
- Fill overhead = 26 of 66 compute cycles — serial twiddle fill is the new bottleneck

Insight: Fill overhead can be eliminated with unique factors preloading.

Exploration 3: Software Twiddle Preload

4.31× Faster

732 → 170 cycles

−15.6% Area

308K → 260K SoC

5-State FSM

Simpler + more accurate

Key Idea

Move twiddle computation from hardware to firmware. Pre-compute full $N/2$ twiddle table in software, write to CSR registers before enabling the accelerator.

Architecture Changes

- Eliminated LOAD_TWIDDLE state and FILL sub-phase → pure butterfly compute
- 32 CSR twiddle registers; SRAM halved 128 → 64 words (−50.5% mem area)
- Single-shot Q12 quantisation replaces chained multiply → better twiddle accuracy

Insight: HW/SW co-design wins: offloading twiddle to firmware reduces both area and cycles.

Exploration 4: Pipelined Butterfly Datapath

~20× Faster

3.1 μ s vs 61 μ s baseline

~60 MHz

5× clock frequency

185 Cycles

3.96× cycle reduction

Key Idea

Break the single-cycle butterfly into a 4-stage micro-pipeline (FETCH $\rightarrow$ MUL1 $\rightarrow$ MUL2 $\rightarrow$ ADD). Each stage is $\sim 1/4$ the combinational depth, enabling $\sim 5\times$ higher clock frequency.

Architecture Changes

- 4 pipeline stages:
FETCH $\rightarrow$ MUL1 (partial products) $\rightarrow$ MUL2 (combine+scale) $\rightarrow$ ADD (writeback)
- 2× parallel lanes, 1-throughput: new butterfly pair every cycle $\rightarrow$ 55 compute cycles
- Speedup = 3.96× fewer cycles $\times$ 5× frequency $\approx 20\times$ latency reduction

Insight: Pipelining unlocks frequency; memory bandwidth remains the binding constraint.

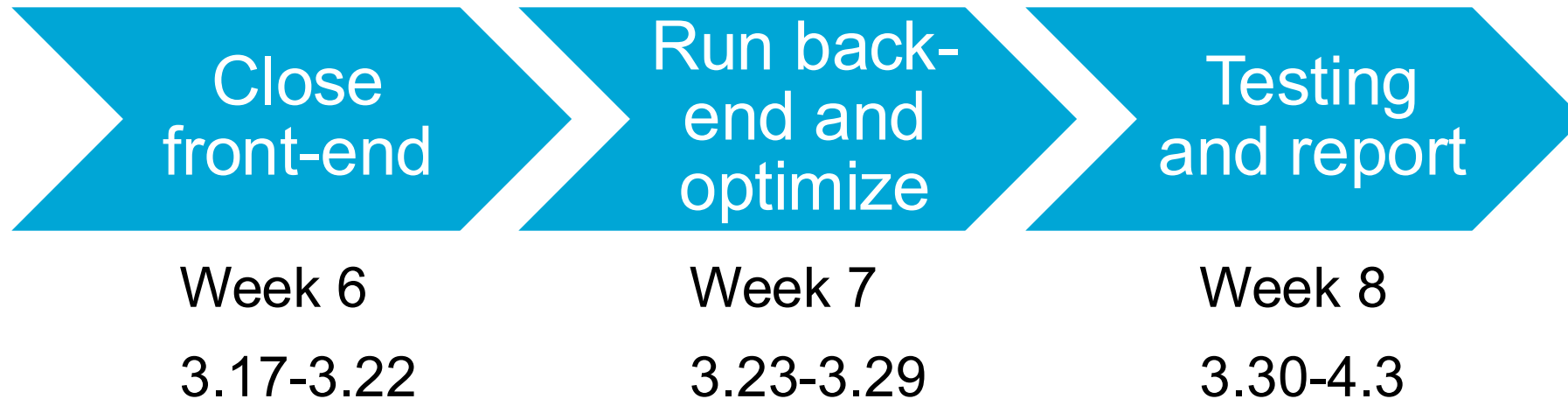
03

Roadmap

Further improvements

- Faster clock for accelerator, different clock domains or dual-edge clocking.
- 2x bandwidth from registers to logic.
- Any stopping point / optimal target, when do we know we have done enough?
- **Design spaces in physical design, what settings / optimizations can we use?**

Next steps





Part II – Energy Efficient

Energy Saving Ideas

- Radix-4
- RFFT
- Clock Gating
- Offload to Register
- Trivial-Twiddle Fast Paths
- Removing Recursive Twiddle Update
- No Twiddle Memory Reads (LUT based)

Radix 2 and 4 Mix

Radix-2 (baseline)	vs	Radix-4 mixed (proposed)
5 stages 5 passes through memory	stages	3 stages 3 passes through memory
80 multiplications per FFT of N=32	mults	64 multiplications per FFT of N=32
Slower (baseline 61 μs) more FSM loop iterations	latency	Faster (target < 61 μs) fewer FSM loop iterations
Higher energy more switching activity	energy	Lower energy less time active = less switching
Simpler hardware 1 butterfly unit (2 inputs)	area	More hardware 1 butterfly unit (4 inputs)

Real FFT (RFFT) Optimization

32 Real Inputs

padded with imag=0

32-pt Complex FFT

Correct but redundant

17 Unique Bins

Optimized RFFT output

Current Design

- Stores each sample as (real, imag) pair
- Imaginary parts initialized to 0
- Runs full 32-pt complex FFT
- Correct, but does not exploit real-input symmetry
- 32 complex outputs; only 17 unique ($X[k] = X^*[32-k]$)

Proposed: 32-pt RFFT

- Same 32-sample input, same frequency resolution
- Pack 32 real samples into 16 complex values
- Run 16-pt complex FFT core (internal implementation trick)
- RFFT post-processing recombination -> 17 unique bins
- Output all 32 bins by reconstructing them from the 17 unique bins using conjugate symmetry
- Reduces arithmetic, memory traffic, and area
- Can be done with 2 radix-4 stages with some post processing

Trivial-Twiddle Fast Paths (1)

- In the FFT butterfly, the baseline computes:
 - $t = w \cdot X_2$, with a **full complex multiplication** for every butterfly.
- With **fastpaths**, if the twiddle w is $+1, -1, +j, -j$, then **no real complex multiply is needed**.
- Instead, compute t with only: **sign flips** and **real/imag swap**
- Examples:
 - $+1 \cdot (a + jb) = a + jb$
 - $-1 \cdot (a + jb) = -a - jb$
 - $+j \cdot (a + jb) = -b + ja$
 - $-j \cdot (a + jb) = b - ja$

Trivial-Twiddle Fast Paths (2)

- Latency does not improve because of **same FSM, same states, and same number of cycles**
- But it can still reduce energy because:
 - less multiplier switching
 - less internal datapath activity
 - lower dynamic power

Removing Recursive Twiddle Update (1)

- In the FFT butterfly flow, the baseline also updates the twiddle recursively:
 - $W \leftarrow W \cdot W_m$
- This means an **extra complex multiplication** is used only for **twiddle progression**, not for the butterfly output itself.
- This recursive update can be removed, and instead, the required twiddle is selected directly from a **precomputed hardcoded lookup table** indexed by (m, k) .
- So the FFT computation is unchanged, but the **twiddle-generation path uses less arithmetic**.

Removing Recursive Twiddle Update (2)

- Latency does not improve because of **same FSM, same states, and same number of cycles**
- But it can still reduce energy because:
 - the **extra complex multiply** for twiddle progression is removed
 - **fewer arithmetic registers toggle** each butterfly
 - **less switching** occurs in the twiddle-generation logic
 - dynamic power can therefore go down

No Twiddle Memory Reads (LUT based) (1)

- In the baseline, the FFT reads the stage twiddle step W_m from memory in:
 - READ_W_M_RE
 - READ_W_M_IM
- These memory reads can be removed, and instead, w_m_re and w_m_im are assigned directly from a **stage-local constant** based on the FFT stage.
- This still works because $W_m = e^{-j2\pi/m}$ depends only on the stage ($m=2,4,8,16,32$), not on base or k .
- So reading W_m from memory or assigning the same fixed-point constant directly is mathematically equivalent.

No Twiddle Memory Reads (LUT based) (2)

- The rest of the butterfly flow is unchanged: same FSM sequence, same recursive twiddle update $w \leftarrow w \cdot W_m$, same butterfly math, and same writeback ordering.
- Latency is not expected to improve because the READ_W_M_* states are still present.
- But it can still reduce energy because:
 - less twiddle-memory read activity
 - fewer address/data toggles
 - less switching on the memory/read path
- So the expected result is:
 - **same runtime**
 - **potentially lower dynamic energy**

Thank you for your attention!

Group 9

18-3-2026

